Assignment 1
Inspection AND Review

Assignment Objectives:

- Inspection of documents (specification, design, code) of given problems.

Prerequisite:

- Create a team –5 or 6 members.

Assignment = In-Class assignment + Take-Home assignment

- **In-Class assignment (**Maxim 50 XP)
  - 40 minutes time for problem solving and delivery
  - TO DO:
    1. Use the *In-class Lab 1 Assignment* from next page.
    2. Inspect the Problem statement, Problem design, Source code and using the Lab01_DefectsChecklist.pdf document, create a review table (use template from Lab01_ReviewForm.docx) for each perspective (Problem statement, Problem design, Source code).
- **Take-Home assignment (**Maxim 100 XP)
  - TO DO:
    1. Inspect the analysis design document and the source code of the application document and using the DesignPhaseDefectsChecklist and ProgramCodingPhaseDefectsChecklist from the Lab01_DefectChecklist.pdf document as a guide, create a Review form. (25XP)
    2. Use one of the tools (CodePanorama, RefDistiller, Gerrit, Code Review, or other review tool that you find available) for the received project assigned by your lab teacher. Prepare a 3 to 5 minutes video with the tool in execution for a designed scenario to illustrate the use of the tool, the findings and the benefits of using it. The video must contain your voice explaining the use of the tool for the specific scenario (in your received project) with comments on the findings. (25XP)

Assignment and Delivery date:

1. Assignment date: laboratory 1
2. Delivery date for **In-Class assignment:** laboratory 1 (max 50 XP)
3. Delivery date (first) **for Take-Home assignment**: laboratory 2 (max 100 XP)
4. Delivery date (last) **for Take-Home assignment**: laboratory 3 (max 50XP)

Turn in:

Delivered and presented in class AND upload on Teams (only one student from the team) after Delivery in class the following archive:

a. Lab1_IC_TH_93X_Name01Name02Name03Name04Name05.zip
   i. document for the solution for the In-Class assignment.
   ii. document with inspection tables (statement, design, code) for the Take-Home assignment A.
   iii. source code for Take-Home assignment B.
   iv. document with inspection tables (statement, design, code) for the Take-Home assignment B.
   v. document with the list of members. No "guest name"! Every person mentioned as a collaborator must contribute.

References

- See Lecture 1.
- See Seminar 1.

In-class Lab 1 Assignment

Problem statement. Given a natural number, compute the minimum number formed with the same digits.

Problem design. The program must have: a subalgorithm that reads the given number, a function that gets the digits of the number; a subalgorithm that builds the minimum number with the same digits; a subalgorithm that prints the new formed number.

Source code.

```
public class CreateMinNumber {
  /**
   * Subalgorithm getDigits1(givenNumber, numberOfDigits, valuesDigits)
   * Extracts digits of givenNumber into valuesDigits (1-based index, LSB first).
   * valuesDigits[1] = least significant digit, valuesDigits[numberOfDigits] = most significant digit.
   * @param givenNumber   the input number
   * @param valuesDigits array to store digits (1-based; size should be >= number of digits + 1)
   * @return numberOfDigits - the count of digits extracted
   */
  static int getDigits1(int givenNumber, int[] valuesDigits) {
    int i = 0;
    while (givenNumber != 0) {
      i = i + 1;
      valuesDigits[i] = givenNumber % 10;
      givenNumber = givenNumber / 10;
    }
    int numberOfDigits = i;
    return numberOfDigits;
  }
  /**
   * Subalgorithm buildNumber1(numberOfDigits, valuesDigits, newNumber)
   * Rebuilds the minumum number with the digits from valuesDigits[1..numberOfDigits].
   * @param numberOfDigits  number of digits
   * @param valuesDigits array of digits (1-based)
   * @return newNumber - the newly formed number
   */
  static int buildNumber1(int numberOfDigits, int[] valuesDigits) {
    int newNumber = 0;
    for (int i = 1; i <= numberOfDigits; i++) {
      newNumber = newNumber * 10 + valuesDigits[i];
    }
    return newNumber;
  }
  public static void main(String[] args) {
    if (args.length < 1) {
      System.err.println("Usage: java DetCifre <number>");
      System.exit(1);
    }
    int givenNumber = Integer.parseInt(args[0]);
    // valuesDigits is 1-based; allocate enough space for digits + index offset
    int[] valuesDigits = new int[20];
    System.out.println("Original number : " + givenNumber);
    // Extract digits
    int numberOfDigits = getDigits1(givenNumber, valuesDigits);
    System.out.println("Number of digits: " + numberOfDigits);
    for (int i = 1; i <= numberOfDigits; i++) {
      System.out.print(valuesDigits[i] + " ");
    }
    System.out.println();
    int newNumber = buildNumber1(numberOfDigits, valuesDigits);
    System.out.println("Minimum number formed with the same digits : " + newNumber);
  }
```

}

Take-Home Lab 1. Assignment A.

Problem statement. Given a natural number, compute the minimum number formed with the same digits.

Problem design. The program must have: a subalgorithm that reads the given number, a function that gets the digits of the number; a subalgorithm that builds the minimum number with the same digits; a subalgorithm that prints the new formed number.

Source code.

```
public class CreateMinNumber2 {
   /*Subalgorithm getDigits2(givenNumber, valuesDigits)
    * Counts the frequency of each digit (0-9) in givenNumber.
    * valuesDigits[d] = how many times digit d appears in givenNumber.
    * @param givenNumber  the input number
    * @param valuesDigits frequency array of size 10 (index 0..9) */
   static void getDigits2(int givenNumber, int[] valuesDigits) {
      // Initialize frequency array
      for (int i = 0; i <= 9; i++) {
         valuesDigits[i] = 0;
      }
      // Count each digit's frequency
      while (givenNumber != 0) {
         int cif = givenNumber % 10;
         valuesDigits[cif] = valuesDigits[cif] + 1;
         givenNumber = givenNumber / 10;
      }
   }
   /*Subalgorithm buildNumber2(valuesDigits, newNumber)
    * Builds the minimum number using the digit frequencies from valuesDigits.
    * Iterates digits 0-9 in ascending order, appending each digit valuesDigits[i] times.
    * @param valuesDigits frequency array of size 10 (index 0..9)
    * @return newNumber - the minimum number formed with the same digits */
   static int buildNumber2(int[] valuesDigits) {
      int newNumber = 0;
      for (int i = 0; i <= 9; i++) {
         for (int j = 1; j <= valuesDigits[i]; j++) {
            newNumber = newNumber * 10 + i;
         }
      }
      return newNumber;
   }
   public static void main(String[] args) {
      if (args.length < 1) {
         System.err.println("Usage: java CreateMinNumber2 <number>");
         System.exit(1);
      }
      int givenNumber = Integer.parseInt(args[0]);
      // valuesDigits[d] = frequency of digit d (index 0..9)
      int[] valuesDigits = new int[10];
      System.out.println("Original number : " + givenNumber);
      // Count digit frequencies
      getDigits2(givenNumber, valuesDigits);
      System.out.println("Digit frequencies:");
      for (int i = 0; i <= 9; i++) {
         if (valuesDigits[i] > 0) {
            System.out.println("  Digit " + i + " appears " + valuesDigits[i] + " time(s)");
         }
      }
      // Build minimum number
      int newNumber = buildNumber2(valuesDigits);
      System.out.println("Minimum number formed with the same digits: " + newNumber);
```

}}

Take-Home Lab 1
Assignment B.

To do:

- **S**plit the tasks between the 5 (6) students.
- 2 students will review the Problem statement and Problem design, and then implement the solution using the pseudocode below.
- 3 students will inspect/review the source code produced by the other 2 students.

Problem statement. Compute the longest sequence such that there exists at most 3 different values.

Problem design: The program must have: a subalgorithm that reads the sequence, a subalgorithm that computes the first position and the length of the longest sequence; a subalgorithm that prints the longest sequence found.

Source code.

```
Subalgorithm ComputeMaxSeq(nrE,valE,pozF,lungF) is:
        Let i←0; pozF←0; lungF←0;
        While i<=nrE execute
                ComputeASequence(i,nrE,valE,lungFinala);
                If lungFinala > lungF
                  Then
                        pozF←I;
                        lungF←lungFinala;
                  Else
                        i←i+1;
                 endIf;
        endWh;
endDetSecvMaxima;

Subalgorithm ComputeASequence (pozStart,nrE,valE,lungFinala) is:
{Input: pozStart, nrE, valE;
 Output: pozStart, lungFinala;}
        Let lungFinala←0;
        While (pozStart<=nrE) si Still3Values(valE(i))  execute
                lungFinala←lungFinala+1;
                pozStart←pozStar+1;
        endWh;
endDetOSecventa;
```